Прямой доступ к памяти (DMA)

Цель работы: ознакомиться со строением контроллера DMA и научиться его использовать для работы с периферией STM32, используя библиотеку HAL.

Введение

В любой микроконтроллерной системе постоянно возникает задача обмена данными между ядром процессора (CPU), оперативной памятью и периферийными устройствами (UART, SPI, АЦП и др.). Существует несколько подходов к реализации этого обмена:

1. **Программный опрос (Polling):** CPU в бесконечном цикле опрашивает статусный регистр периферийного устройства, ожидая установки флага готовности данных. Этот метод крайне неэффективен, так как 100% времени CPU занято ожиданием, что блокирует выполнение других задач.
2. **Прерывания (Interrupts):** Периферийное устройство генерирует сигнал прерывания по завершении операции (например, прием одного байта). CPU приостанавливает текущую задачу, переходит в обработчик прерывания, выполняет операцию копирования данных (например, из регистра данных UART в память) и возвращается к основной программе. Этот подход значительно эффективнее опроса, однако при высокой интенсивности потока данных (тысячи байт в секунду) частые переключения контекста создают существенную нагрузку на CPU, снижая общую производительность системы.

Оба метода требуют прямого участия CPU в каждой операции пересылки элементарных единиц данных (байт или слово). Это становится "узким местом" в системах, требующих одновременной обработки больших массивов данных и выполнения сложных вычислительных задач.

Концепция прямого доступа к памяти (DMA)

Для решения проблемы загрузки CPU был разработан **контроллер прямого доступа к памяти (Direct Memory Access, DMA)**.

DMA — это специализированный аппаратный модуль, предназначенный для выполнения операций копирования данных между областями памяти и регистрами периферийных устройств **без участия центрального процессора**.

CPU инициирует транзакцию, настраивая контроллер DMA: указывает источник данных, приемник, объем данных и параметры передачи. После запуска контроллер DMA самостоятельно управляет процессом, запрашивая доступ к системной шине и пересылая данные. CPU в это время может выполнять другие задачи или находиться в режиме

пониженного энергопотребления. По завершении всей транзакции DMA может сгенерировать прерывание, чтобы уведомить CPU.

Ключевые преимущества использования DMA:

- **Разгрузка CPU:** Высвобождение процессорного времени от рутинных операций пересылки данных для выполнения алгоритмической части программы.
- **Высокая производительность:** Передача данных происходит на аппаратном уровне с максимальной скоростью, доступной для системной шины.
- **Снижение энергопотребления:** CPU может находиться в энергосберегающих режимах (Sleep), пока DMA выполняет фоновую передачу данных.

Архитектура DMA в STM32

Контроллер DMA в STM32 является мастером шины (bus master), что позволяет ему инициировать транзакции чтения/записи в системной матрице шин наравне с ядром Cortex-M.

Основные компоненты:

- **Каналы (Channels) / Потоки (Streams):** Контроллер DMA содержит несколько независимых аппаратных ресурсов для конфигурации транзакций.
  - В сериях STM32F1/L1 они называются **каналами**. Каждый канал жестко или с помощью мультиплексора связан с запросами от определенной периферии.
  - В более производительных сериях (STM32F4/F7/H7) архитектура сложнее и включает несколько контроллеров DMA, каждый из которых имеет по несколько **потоков**. В свою очередь, каждый поток через мультиплексор (выбираемый как **канал**) может быть сопоставлен с запросом от одного из нескольких периферийных устройств. Каждый поток представляет собой отдельный, полностью настраиваемый ресурс.
- **Арбитр (Arbiter):** Встроенный аппаратный модуль, который управляет доступом к системной шине при одновременных запросах от нескольких потоков/каналов DMA или от CPU. Арбитраж происходит на основе программно задаваемых уровней приоритета (низкий, средний, высокий, очень высокий), что позволяет гарантировать своевременное обслуживание критически важных потоков данных.

Направления передачи:

- **Периферия -> Память** (Peripheral-to-Memory): Сбор данных с периферии (например, АЦП) в буфер в ОЗУ.

- **Память -> Периферия** (Memory-to-Peripheral): Отправка данных из буфера в ОЗУ на периферийное устройство (например, передача пакета данных через SPI).
- **Память -> Память** (Memory-to-Memory): Высокоскоростное копирование блоков данных внутри ОЗУ, аппаратная реализация функции **memcpy().**

Ключевые параметры конфигурации:

- **Инкремент адреса (Address Increment):** Возможность автоматического увеличения адреса источника и/или приемника после каждой пересылки элемента данных. Для буфера в памяти инкремент обычно включен, для регистра данных периферии (который имеет фиксированный адрес) — выключен.
- **Размер данных (Data Size):** Определение размера элементарной единицы данных для транзакции: 8 бит (байт), 16 бит (полуслово) или 32 бита (слово). Должен соответствовать формату регистра периферии и типу данных в буфере.
- **Кольцевой режим (Circular Mode):** Режим, при котором после достижения конца буфера DMA автоматически возвращается к его началу, продолжая запись или чтение. Этот режим незаменим для организации непрерывной потоковой обработки данных, например, при приеме аудиокодека или данных с АЦП.

Регистры контроллера DMA

Библиотека HAL абстрагирует прямое взаимодействие с регистрами, однако понимание их назначения необходимо для отладки и глубокого анализа работы контроллера. Рассмотрим основные регистры на примере одного канала/потока:

- **Регистры статуса и сброса флагов (DMA_ISR/DMA_LISR/DMA_HISR, DMA_IFCR/DMA_LIFCR/DMA_HIFCR):** Содержат флаги событий: завершение передачи (TCIF), завершение половины передачи (HTIF), ошибка передачи (TEIF) и другие. Записью в регистр IFCR производится сброс флагов.
- **Регистр конфигурации (DMA_SxCR или DMA_CCRx):** Основной регистр для настройки параметров потока/канала. Содержит битовые поля для установки направления передачи (DIR), включения кольцевого режима (CIRC), инкремента адресов (PINC, MINC), размеров данных (PSIZE, MSIZE), приоритета (PL) и выбора канала-мультиплексора (CHSEL, для серий F4/F7).
- **Регистр количества данных (DMA_SxNDTR или DMA_CNDTRx):** Задается общее количество единиц данных для передачи. Регистр является счетчиком с обратным отсчетом.

- **Регистр адреса периферии (DMA_SxPAR или DMA_CPARx):** Содержит фиксированный адрес регистра данных периферийного устройства.
- **Регистр адреса памяти (DMA_SxM0AR или DMA_CMARx):** Содержит начальный адрес буфера в оперативной памяти.

Прямая конфигурация этих регистров требует внимательного изучения Reference Manual и является трудоемкой задачей, которую упрощает использование HAL.

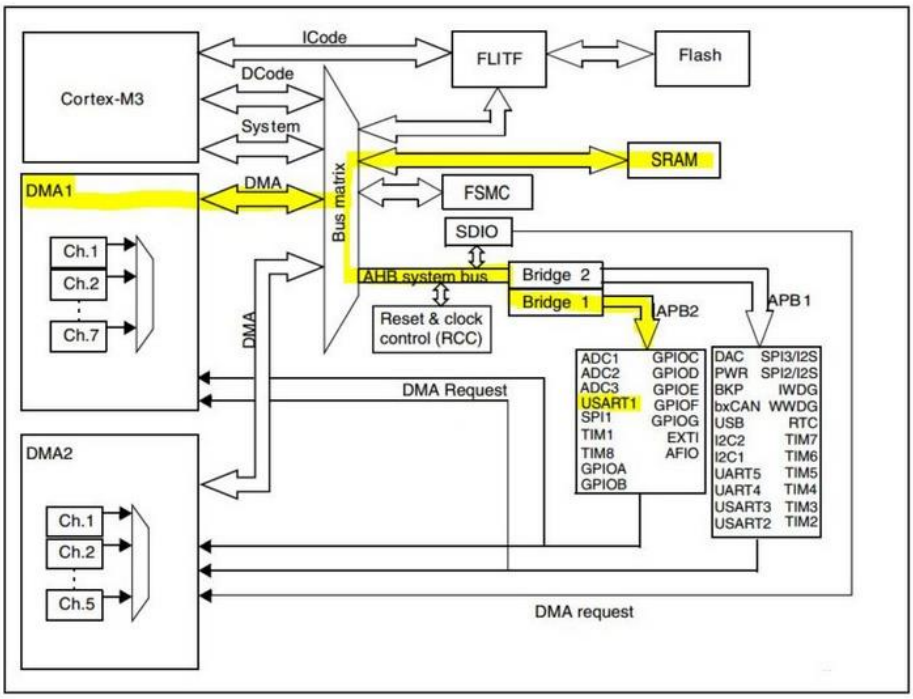
Использование DMA с библиотекой HAL

Рассмотрим алгоритм настройки и запуска DMA на примере передачи данных через UART.

Последовательность действий:

1. **Тактирование:** Включается тактирование контроллера DMA и соответствующего периферийного устройства.
2. **Конфигурация периферии:** Настраивается периферийное устройство (например, UART) в режиме, поддерживающем DMA.
3. **Конфигурация DMA:** С помощью CubeMX или вручную инициализируется структура *DMA_HandleTypeDef*, в которой задаются все параметры из пункта 3 (направление, инкремент, режим и т.д.). Вызывается функция *HAL_DMA_Init()*.
4. **Связывание DMA и периферии:** Экземпляр DMA связывается с дескриптором периферии. Например, *hdma_usart1_tx* присваивается полю *huart1.hdmatx*.

Процесс передачи данных с DMA (на примере USART1 -> SRAM)



Эта схема иллюстрирует, как DMA в микроконтроллере STM32 позволяет передавать данные между периферией и памятью **без** участия ЦПУ, освобождая его для других задач.

- **Cortex-M3 (ЦПУ):** "Мозг" системы. Он выполняет код, но в контексте DMA его главная роль — **один раз настроить** контроллер DMA, указав ему, *откуда* (источник), *куда* (назначение) и *сколько* данных нужно передать.
- **DMA1 / DMA2:** Два независимых контроллера прямого доступа к памяти. Они являются "мастерами шины" (bus masters), так же, как и ЦПУ. Это означает, что они могут самостоятельно инициировать чтение и запись в память или периферию.
- **Bus matrix (Матрица шин):** Центральный коммутатор ("диспетчер"). Он управляет доступом к ресурсам (SRAM, Flash, периферийные шины) от нескольких "мастеров" (Cortex-M3, DMA1, DMA2). Матрица решает, кто в данный момент получает доступ, позволяя ЦПУ и DMA работать параллельно.
- **Периферия (USART1, ADC, SPI и т.д.):** Устройства ввода-вывода. В данной схеме они подключены к более медленным шинам APB1 и APB2.
- **SRAM / Flash:** Оперативная память и флеш-память, подключенные к высокоскоростной шине.

Проследим типичный сценарий: получение данных через USART1 (например, из GPS-модуля) и сохранение их в SRAM.

1. **Настройка (роль ЦПУ):**
   - Cortex-M3 сначала настраивает USART1 (на шине APB2) на прием данных.
   - Затем Cortex-M3 настраивает один из каналов DMA1 (например, Ch.1). Он "говорит" DMA: "Когда USART1 попросит, возьми данные из его регистра данных и положи их вот по этому адресу в SRAM. Повторяй, пока не передашь 100 байт".
   - После этой настройки ЦПУ **свободен** и может выполнять другие задачи (например, обсчитывать математику или моргать светодиодом).
2. **Запрос от периферии (Запуск):**
   - В USART1 поступает первый байт данных.
   - USART1 (выделен желтым), вместо того чтобы прерывать ЦПУ, посылает сигнал DMA Request (Запрос DMA) напрямую контроллеру DMA1.
3. **Работа DMA (Передача без ЦПУ):**
   - DMA1 (выделен желтым) получает этот запрос.
   - DMA1 становится "мастером" на шине и запрашивает доступ у Bus matrix.

- **Чтение из периферии:** DMA1 обращается к Bus matrix, которая направляет его через AHB system bus, далее через Bridge 1 (выделен желтым) на шину APB2, чтобы прочитать байт данных из USART1.
- **Запись в память:** DMA1 немедленно снова обращается к Bus matrix (выделен желтым) и записывает этот байт напрямую в SRAM (выделен желтым).
- Этот цикл (Запрос -> Чтение -> Запись) повторяется для каждого нового байта, приходящего на USART1, автоматически.

4. **Параллельная работа:**
   - Пока DMA1 занят этой пересылкой, Cortex-M3 продолжает выполнять свой код.
   - Единственный момент, когда ЦПУ может приостановиться, — это если он и DMA1 попытаются обратиться к SRAM **одновременно**. В этом случае Bus matrix ("диспетчер") справедливо распределит доступ, что все равно во много раз быстрее, чем если бы ЦПУ сам копировал каждый байт.

Задание

1. Реализуйте генерацию ШИМ-сигнала с канала таймера TIM на вашей плате. Частоту ШИМ-сигнала выберите на своё усмотрение.
   Затем реализуйте управление скважностью ШИМ-сигнала с канала таймера TIM через DMA. Для этого заведите массив со значениями регистра CCR (массив «скважностей»), и передайте его в функцию генерации ШИМ через DMA.
   В массиве скважностей должны последовательно лежать следующие варианты скважности ШИМ-сигнала:
   [0]: 20% заполнения.
   [1]: 40% заполнения.
   [2]: 60% заполнения.
   [3]: 70% заполнения.
   [4]: 90% заполнения.
   [5]: 70% заполнения.
   [6]: 60% заполнения.
   [7]: 40% заполнения.
   [8]: 20% заполнения.
   Представьте сгенерированный сигнал на осциллографе.
   -ИЛИ-
   Представьте пример, в котором по данному массиву медленно загорается и затухает светодиод. Можно взять как светодиод на плате, так и поставить внешний светодиод.

2. Реализуйте подачу переменного сигнала на АЦП и считайте его с канала АЦП в режиме DMA. Выберите форму (синус, треугольник, пила), частоту, амплитуду сигнала на своё усмотрение.
   Продемонстрируйте полученные значения сигнала с АЦП. Можно продемонстрировать скриншотом массива значений (в вольтах), получаемых с АЦП.

3. Реализуйте пример из лабораторной работы №6 с передачей строк между двумя платами. В качестве протокола передачи возьмите SPI или I2C.
   Передача должна быть реализована через DMA!

4. В отчете представьте результаты выполненных шагов 1-3. Приложите к отчёту файл(-ы) main.c, где реализовано выполнение заданий 1-3.